

Kapitel 4 · In MySQL speichern

Fahrradverleih · Voraussetzung: Kapitel 1–3

Das lernen Sie

Sie verbinden Python mit der vorhandenen Datenbank und speichern Fahrraddaten mit einem SQL-Statement. Dabei führen Sie die Bausteine der ersten drei Kapitel zusammen.

1. Vorbereitung

Allgemeine Grundlagen: Python-Programm, Datenbanktreiber und Datenbankserver übernehmen verschiedene Aufgaben. Das Programm sammelt und verarbeitet Werte; der Treiber übermittelt Befehle; der Server verwaltet Tabellen und Datensätze.

Anwendung: Fahrradverleih

Der lokale MySQL-/MariaDB-Server muss laufen. Die Datenbank fahrradverleih mit der Tabelle fahrraeder ist über den Import der Datei fahrradverleih.sql angelegt worden.

Das Python-Paket mit dem Treiber für den MySQL-/MariaDB-Server muss installiert werden: in **VSC**: im Terminal eingeben: `pip install mysql-connector-python`

Thonny: *Werkzeuge* → *Verwaltete Pakete* → Paketname eingeben und installieren lassen.

pip (Pip Installs Packages): Es ist das Werkzeug (der Paketmanager), der die Verbindung zum Internet herstellt, auf PyPI nach dem gewünschten Paket sucht, es herunter lädt und installiert.

PyPI (Python Package Index): Es ist die Website und Datenbank im Internet, auf der Hunderttausende von Python-Paketen (Bibliotheken, Tools, Plugins) von Entwicklern weltweit gespeichert sind.

2. Verbindung und Cursor

Allgemeine Grundlagen: Eine Verbindung ist eine Sitzung mit dem Datenbankserver. Die Verbindungsparameter bestimmen Ziel und Anmeldung: host bezeichnet den Server, user und password den Zugang, database die verwendete Datenbank. localhost bezeichnet hier den eigenen Rechner. Ein Cursor gehört zu einer Verbindung und dient zum Ausführen von SQL-Anweisungen.

Anwendung: Fahrradverleih

```
import mysql.connector
from datetime import date
connection = mysql.connector.connect(
    host="localhost",
    user="root",
    password="",
    database="fahrradverleih",
)
cursor = connection.cursor()
```

import lädt den Treiber. connect() öffnet eine Verbindung; die benannten Argumente geben Server, Benutzer, Passwort und Datenbank an.

connection hält die Verbindung. Ein Cursor ist das Werkzeug, mit dem Sie SQL-Befehle über diese Verbindung ausführen. Er entsteht durch die Methode connection.cursor().

Kapitel 4 · SQL und Python-Werte

3. Welche Spalten werden gefüllt?

Allgemeine Grundlagen: Eine Tabelle besteht aus Spalten mit festgelegten Datentypen und aus Zeilen, die einzelne Datensätze darstellen. Beim INSERT bestimmt die Spaltenliste, welche Bedeutung die übergebenen Werte haben. Automatisch erzeugte Primärschlüsselwerte müssen nicht selbst eingegeben werden.

Allgemeines Beispiel: Bei der gedachten Spaltenliste (bezeichnung, anzahl) passen die Werte ("Heft", 3) zusammen. (3, "Heft") vertauscht die Bedeutung der Angaben.

Anwendung: Fahrradverleih

fahrraeder enthält fahrradNr (automatisch hochgezählte ID), rahmenNr (Text), tagesmietpreis (Zahl), anschaffungsdatum (Datum) und artikelNr (Ganzzahl). fahrradNr lassen Sie beim Einfügen weg, weil AUTO_INCREMENT den Wert erzeugt. Die anderen vier Werte stammen aus Ihren Eingaben.

Testen Sie die MySQL-Verbindung und die Dateneingabe über Python. Führen Sie dazu das Programm *MySQL_Test.py* aus. Erstellen Sie die benötigte Datenbank, indem Sie die Datei *fahrradverleih.sql* in phpmyadmin importieren.

4. Statement und Daten getrennt übergeben

Allgemeine Grundlagen: Das SQL-Statement beschreibt den Befehl, während das Datentupel die konkreten Werte enthält. Ein Tupel bündelt Werte in einer festen Reihenfolge. `execute()` erhält beides getrennt.

Merksatz – Platzhalter: %s steht beim MySQL-Connector für einen Wert, auch für Zahlen und Datum. Werden Daten von Benutzern eingegeben, die in eine Datenbank gespeichert werden, müssen Platzhalter eingesetzt werden (Schutz vor SQL-Injections).

`execute()`: Ein Aufruf führt das Statement mit den zugehörigen Werten aus. Eine dauerhafte Bestätigung erfolgt gesondert durch `commit()`.

Anwendung: Fahrradverleih

```
sql = """
INSERT INTO fahrraeder
    (rahmenNr, tagesmietpreis, anschaffungsdatum, artikelNr)
VALUES (%s, %s, %s, %s);
"""

daten = (rahmenNr, tagesmietpreis, anschaffungsdatum, artikelNr)
cursor.execute(sql, daten)
```

Die dreifachen Anführungszeichen begrenzen einen mehrzeiligen Python-Text. `INSERT INTO` nennt die Tabelle und die zu befüllenden Spalten. `VALUES` enthält hier vier Platzhalter für die vier Werte.

`daten` ist ein Tupel: Mehrere Werte werden durch Kommas in einer festen Reihenfolge zusammengefasst. Die Reihenfolge muss zur Spaltenliste passen: erst Rahmennummer, dann Preis, Datum und Artikelnummer.

`execute(sql, daten)` führt das Statement mit diesen Werten aus. Der Treiber verwendet `%s` für alle vier Datentypen, auch für Zahlen und Datum. Setzen Sie keine Anführungszeichen um `%s`. Übergeben Sie die Werte als zweiten Parameter, der Treiber übernimmt die passende Behandlung der Werte.

5. Speichern und aufräumen

Nach einem erfolgreichen `INSERT` bestätigt `commit()` die Änderung. `close()` gibt die verwendeten Ressourcen frei.

Anwendung: Fahrradverleih

```
connection.commit()
cursor.close()
connection.close()
```

`commit()` bestätigt die Änderung und speichert sie dauerhaft. Danach schließen Sie Cursor und Verbindung mit `close()`. Bei dieser Verbindung werden Änderungen nicht automatisch bestätigt. Geben Sie eine Erfolgsmeldung erst nach `commit()` aus.

Jeder erfolgreiche Programmlauf fügt einen neuen Datensatz hinzu.

Kapitel 4 · Pflichtübung / Gesamtaufgabe

Auftrag: Ein Fahrrad speichern

Erstellen Sie `uebung_04.py` als zusammenhängendes Programm. Nutzen Sie ausschließlich die Bausteine der vier Kapitel.

1. Importieren Sie `mysql.connector` und `date`. Erfassen Sie eine Rahmennummer mit der `if-/while`-Prüfung aus Kapitel 3 (ein bis sechs Zeichen).
2. Fragen Sie den Tagesmietpreis ab, ersetzen Sie ein Komma durch einen Punkt und wandeln Sie den Text in `float` um.
3. Fragen Sie Jahr, Monat und Tag als `int` ab und erzeugen Sie `anschaffungsdatum` mit `date()`. Fragen Sie `artikelNr` als `int` ab.
4. Öffnen Sie die Verbindung zu `fahrradverleih` mit den Zugangsdaten des MariaDB-Servers und erzeugen Sie einen Cursor.
5. Formulieren Sie `INSERT INTO` für die vier Eingabespalten mit vier `%s`-Platzhaltern. Erstellen Sie das Tupel `daten` in derselben Reihenfolge und rufen Sie `execute(sql, daten)` auf.
6. Bestätigen Sie die Änderung mit `commit()`, schließen Sie Cursor und Verbindung und geben Sie „Fahrrad gespeichert.“ aus.

Prüfen in der Übungsdatenbank

Notieren Sie zunächst die Zeilenanzahl der Tabelle, beispielsweise in `phpMyAdmin`. Starten Sie dann Ihr Programm mit den Eingaben `AB1234`, `12,50`, `2026`, `9`, `8` und `4711`. Aktualisieren Sie anschließend die Tabellenansicht.

Erwartung: Genau eine neue Zeile mit `rahmenNr` `AB1234`, `tagesmietpreis` `12.50` (je nach Anzeige auch `12.5`), `anschaffungsdatum` `2026-09-08` und `artikelNr` `4711`. `fahrradNr` wird automatisch vergeben; Sie geben diese Nummer nicht ein.

Testen Sie in einem weiteren Lauf zuerst eine leere und dann eine siebenstellige Rahmennummer. Erst nach einer gültigen Eingabe dürfen die übrigen Fragen folgen. Nach dem vollständigen erfolgreichen Lauf gibt es genau eine weitere Zeile, keine Zeile für die verworfenen Eingaben.

Erledigt, wenn ...

Eingabeprüfung und Speicherung funktionieren. Sie können erklären, warum vier Platzhalter und vier Werte benötigt werden und weshalb `execute()` und `commit()` zwei verschiedene Schritte sind.

Zusatz:

7. Passen Sie das Python-Script zur Dateneingabe an die Datenbank aus dem DB-Projekt an.

Zum Nachlesen

`execute()`: <https://dev.mysql.com/doc/connector-python/en/connector-python-api-mysqlicursor-execute.html>

`commit()`: <https://dev.mysql.com/doc/connector-python/en/connector-python-api-mysqlconnection-commit.html>